

UNIVERSIDAD DE LOS ANDES
Facultad de Ingeniería y Ciencias Aplicadas
Arquitectura de computadores

Proyecto 2 - Juego en Assembly

Grupo 4:
Estudiante 1: Christian Georgi
Estudiante 2: Diego Quinteros
Estudiante 3: Felipe Riveros

Profesor de Cátedra: Jorge Gómez Mir

Octubre de 2026

Índice

1. Introducción **3**

2. Arquitectura general del sistema **5**

2.1. Arquitectura base 5

2.2. Integración del juego 5

2.3. Flujo de ejecución y acceso a periféricos 5

3. Desarrollo y modificaciones de hardware **7**

3.1. Nuevo módulo superior: `game_top.v` 7

3.2. Contador de ciclos: `pochoco_periph.v` 7

3.3. Modificación del reset: `pochoco_soc.v` 8

4. Implementación del juego en Assembly **9**

4.1. Estructura general del programa 9

4.2. Inicialización y control de LEDs y botones 11

4.3. Temporización inicial 11

4.4. Selección pseudoaleatoria 13

4.5. Validación de respuestas y avance 13

4.6. Medición y conversión del tiempo de reacción 14

4.7. Visualización de resultados 15

4.8. Conteo de rondas y cálculo del promedio 15

5. Desarrollo del Assembler **16**

5.1. Objetivo y procesamiento del archivo 16

5.2. Sintaxis admitida e inmediatos 16

5.3. Instrucciones soportadas 17

5.4. Procesamiento de etiquetas y saltos 17

5.5. Codificación de las palabras de 32 bits 18

5.6. Ejemplo de traducción Assembly a código máquina 18

5.7. Validación y pruebas del assembler 19

6. Integración y proceso de compilación **20**

6.1. Modificaciones del Makefile 20

6.2. Generación automática, síntesis y programación 20

6.3. Interacción entre software y hardware 20

7. Validación y pruebas **21**

7.1. Validación del funcionamiento general 21

7.2. Validación de la medición del tiempo 21

7.3. Validación de las diez rondas y promedio final 22

7.4. Validación de la integración hardware-software 22

8. Declaración de uso de IA **23**

1. Introducción

El proyecto se enfoca en diseñar y desarrollar un juego de reflejos programado en Assembly para ejecutarlo sobre Pochoco SoC en una FPGA Lattice iCE40 HX1K. El sistema se implementa mediante instrucciones almacenadas en memoria. El procesador ejecuta estas instrucciones y se comunica con los LEDs, botones y displays a través de los periféricos mapeados en memoria.

Durante cada intento se encienden los cuatro LEDs durante un intervalo superior a tres segundos. Luego se selecciona uno de ellos y el jugador debe presionar el botón correspondiente. Si la respuesta es correcta, se obtiene el tiempo de reacción, se muestra en décimas de segundo y se incorpora al acumulado. La partida termina al completar diez respuestas correctas; posteriormente, se presenta un promedio entero de los tiempos registrados.

Objetivo general

Integrar un programa en Assembly con una arquitectura de procesador existente, incorporando los recursos de temporización y las herramientas necesarias para ejecutar el juego en la FPGA.

Objetivos específicos

- Implementar el control de rondas y respuestas.
- Desarrollar un assembler que traduzca el programa a palabras de 32 bits.
- Incorporar un contador accesible por software.
- Adaptar el arranque y el módulo superior.
- Conectar la generación del programa con el flujo de síntesis y programación.

Este informe se estructura de la siguiente manera: en primer lugar, se presenta la arquitectura general del sistema. Posteriormente, se describen el desarrollo y las modificaciones de hardware. Luego, se expone la implementación del juego en Assembly, el desarrollo del assembler y la integración con el proceso de compilación. Finalmente, se presentan las validaciones y pruebas realizadas.

2. Arquitectura general del sistema

2.1. Arquitectura base

Para el proyecto se utilizó la FPGA Lattice iCE40 HX1K con un procesador Espino Core de 32 bits, con el objetivo de implementar un juego de reflejos que mide el tiempo de reacción del usuario al presionar el botón correspondiente al LED seleccionado de manera pseudoaleatoria.

La arquitectura base incluye búsqueda de instrucciones, decodificación, banco de registros, ALU, unidad de carga y almacenamiento y control de esperas. El registro `x0` entrega el valor constante cero, `x1` se utiliza como estado pseudoaleatorio y `x2` se reserva como dirección base de los periféricos.

La memoria `pochoco_ram` es unificada y posee un puerto A para instrucciones y un puerto B para datos. Aunque su parámetro por defecto es de 2048 palabras, el SoC la instancia con `NumWords=512`; `game_top` no sobrescribe ese valor. Por lo tanto, se configuran 512 palabras de 32 bits, equivalentes a 2048 bytes.

2.2. Integración del juego

El módulo `game_top` instancia un único `pochoco_soc` con `MemFile="sw/game.hex"`. El SoC conecta el núcleo, la RAM, los periféricos de la placa y el esclavo SPI. La interfaz SPI permanece conectada porque pertenece a la arquitectura base, pero el programa del juego no la utiliza. No se utiliza un segundo procesador ni una máquina de estados adicional para controlar las rondas.

2.3. Flujo de ejecución y acceso a periféricos

El assembler produce una palabra hexadecimal por instrucción. Durante la elaboración del diseño, `$readmemh(MemFile, mem)` inicializa la RAM con esas palabras. El contenido de `game.hex` se incorpora al diseño sintetizado; la FPGA no abre un archivo de texto en cada arranque. Al liberarse el reset, el núcleo comienza su ejecución desde `BootAddr=0x00000000`.

La etapa de búsqueda solicita la palabra correspondiente al contador de programa. La RAM registra la lectura y el decodificador obtiene el opcode, los registros y los inmediatos. La ALU ejecuta operaciones aritméticas, lógicas o comparaciones, mientras que la unidad de memoria realiza operaciones `lw` y `sw`. El controlador detiene el avance cuando el banco de registros o una carga requieren espera. Por este motivo, un ciclo del contador de reloj no equivale necesariamente a una instrucción ejecutada.

Los periféricos utilizados por el juego se encuentran mapeados a partir de la dirección `0x80000000`. En los periféricos, `addr_i[7:2]` determina el registro interno.

Dirección usada	Acceso	Comportamiento
`0x80000000`	Escritura	Actualiza los displays de 7 segmentos.
`0x80000004`	Escritura	Actualiza los cuatro LEDs.
`0x80000008`	Lectura	Entrega el estado de los cuatro botones.
`0x8000000C`	Lectura	Entrega una muestra del contador de ciclos.

Cuadro 1: Direcciones MMIO utilizadas por `game.s`.

Fuente: Elaboración propia

3. Desarrollo y modificaciones de hardware

3.1. Nuevo módulo superior: `game_top.v`

Para el proyecto se creó un nuevo módulo superior llamado `game_top.v`, encargado de instanciar el Pochoco SoC y conectarlo con las entradas y salidas físicas de la Nandland Go Board. Este módulo conserva las señales de reloj, botones, LEDs, displays de 7 segmentos y SPI utilizadas por la arquitectura original.

El cambio principal consiste en indicar que la memoria del Pochoco SoC debe inicializarse con el archivo `sw/game.hex`, el cual contiene el código máquina generado a partir del programa `game.s`.

De esta forma, al sintetizar `game_top.v`, el procesador Espino Core dispone en memoria de las instrucciones correspondientes al juego y puede ejecutarlas directamente al iniciar el sistema. La lógica principal del juego no se implementa en Verilog dentro de `game_top.v`, sino en el programa Assembly `game.s`, que posteriormente es convertido a `game.hex` mediante el assembler desarrollado para el proyecto.

3.2. Contador de ciclos: `pochoco_periph.v`

Para implementar las temporizaciones y medir el tiempo de reacción del jugador se agregó un contador de ciclos de 32 bits al módulo de periféricos `pochoco_periph.v`.

El contador comienza en cero después del reset y aumenta en una unidad en cada ciclo de reloj. Para que el procesador pueda acceder a él se agregó una nueva posición al mapa de memoria de los periféricos. El contador puede leerse desde la dirección `0x8000000C` mediante una instrucción `lw`.

El programa no necesita reiniciar el contador para realizar una medición. En su lugar, guarda el valor al comenzar la medición y posteriormente obtiene una segunda lectura. La cantidad de ciclos transcurridos se calcula como:

$$N_{\text{ciclos}} = C_{\text{final}} - C_{\text{inicial}}$$

Este contador se utiliza principalmente para implementar la espera inicial de cada ronda, medir el tiempo de reacción del jugador y aportar variación a la selección pseudoaleatoria del LED objetivo.

3.3. Modificación del reset: `pochoco_soc.v`

Durante las primeras pruebas realizadas en la FPGA se observó que el arranque del sistema no era siempre consistente. En algunas programaciones el procesador comenzaba a ejecutar correctamente, mientras que en otras los LEDs no presentaban el comportamiento esperado.

Para solucionar este problema se aumentó el tiempo durante el cual el SoC permanece en estado de reset después del encendido. Para ello se utilizó un contador de 16 bits en el circuito de *Power-On Reset*. Mientras el contador no alcanza su valor máximo, la señal `rst_ni` permanece en nivel bajo y mantiene los módulos del sistema en estado de reset.

Considerando una frecuencia de reloj de 25 MHz, el tiempo aproximado antes de liberar el reset es:

$$t_{\text{reset}} = \frac{65535}{25 \times 10^6} \approx 2{,}62\,\text{ms}$$

Una vez alcanzado el valor máximo, el contador deja de incrementarse y la señal `rst_ni` pasa a nivel alto, permitiendo que el procesador y los demás módulos comiencen su funcionamiento normal. Después de realizar esta modificación, el arranque observado durante las pruebas en la FPGA fue más consistente.

4. Implementación del juego en Assembly

4.1. Estructura general del programa

El programa fue desarrollado en Assembly utilizando instrucciones compatibles con el procesador Espino Core. Su funcionamiento se organiza mediante una secuencia de instrucciones, bucles y saltos condicionales e incondicionales que permiten controlar las distintas etapas del juego.

El programa utiliza 85 instrucciones y 26 etiquetas, empleadas para identificar las diferentes secciones y controlar el flujo de ejecución. No requiere interrupciones ni subrutinas, ya que el funcionamiento se controla mediante los registros del procesador y las instrucciones de salto.

Para almacenar los valores necesarios durante la ejecución se utilizan distintos registros del procesador, que permiten guardar las mediciones del contador de ciclos, los tiempos de reacción, la cantidad de respuestas correctas y los resultados de las operaciones.

Registro	Uso principal
`x0`	Constante cero y destino descartado de saltos incondicionales.
`x1`	Estado pseudoaleatorio; recibe muestras del contador mediante XOR.
`x2`	Dirección base MMIO: `0x80000000`.
`x3`	Lectura de botones, selección enmascarada o nibble de unidades.
`x4`	Máscara de LED; posteriormente, resultado codificado para display.
`x5`	Umbral temporal, máscara, divisor de conversión o divisor 10.
`x6`	Muestra inicial del contador; se reutiliza como constante 10.
`x7`	Muestra posterior del contador.
`x8`	Diferencia de ciclos; residuo durante la conversión.
`x9`	Número de respuestas correctas.
`x10`	Límite de diez aciertos al finalizar cada ronda válida.
`x11`	Suma de tiempos enteros en décimas; residuo al promediar.
`x12`	Tiempo de una respuesta expresado en décimas.
`x13--x15`	No utilizados por el juego.

Cuadro 2: Asignación y reutilización de registros.

Fuente: Elaboración propia

4.2. Inicialización y control de LEDs y botones

La instrucción `lui x2, 0x80000` construye la dirección base de los periféricos. Luego se inicializan en cero los registros `x9` y `x11`. El bucle `wait_start_press` lee los botones hasta detectar un valor distinto de cero. En ese instante se toma una muestra del contador de ciclos como semilla inicial almacenada en `x1`. Antes de comenzar la primera ronda, `wait_start_release` espera que todos los botones vuelvan a cero.

En `start_round`, la constante 15 se escribe en los LEDs. Su representación binaria, `1111`, activa los cuatro LEDs. Posteriormente, el programa escribe únicamente una de las máscaras `1`, `2`, `4` u `8`, correspondientes a los cuatro LEDs disponibles.

Una respuesta incorrecta reinicia la ronda actual, mientras que una respuesta correcta actualiza el tiempo y el número de aciertos, espera la liberación del botón y decide si corresponde iniciar otra ronda o calcular el promedio final.

4.3. Temporización inicial

Para una espera de duración t, el número ideal de ciclos es:

$$N_{\text{ciclos}} = f_{\text{reloj}} \cdot t$$

El código guarda una muestra inicial del contador en `x6`, carga el umbral temporal en `x5` y consulta muestras sucesivas hasta que la diferencia alcanza el valor programado.

```
lw    x6, 12(x2)
lui   x5, 0x04C00

wait_3s:
    lw    x7, 12(x2)
    sub   x8, x7, x6
    bge   x8, x5, choose_target
    jal   x0, wait_3s
```

La instrucción `lui` coloca su inmediato en los veinte bits superiores y completa con doce ceros. Por lo tanto:

$$N_{\text{espera}} = 0x04C00 \cdot 2^{12} = 79\,691\,776$$

Considerando una frecuencia de 25 MHz:

$$t_{\text{espera}} = \frac{79\,691\,776}{25\,000\,000} \approx 3{,}19\,\text{s}$$

De esta forma se cumple el requisito de mantener los cuatro LEDs encendidos durante al menos tres segundos.

4.4. Selección pseudoaleatoria

En `choose_target` se lee nuevamente el contador de ciclos y se combina con el estado almacenado en `x1` mediante XOR. La máscara `0x00300000` selecciona los bits 21 y 20.

```
lw    x7, 12(x2)
xor   x1, x1, x7
lui   x5, 0x00300
and   x3, x1, x5
```

Los cuatro valores posibles se utilizan para elegir uno de los cuatro LEDs. Después de una respuesta correcta, el estado almacenado en `x1` vuelve a modificarse mediante XOR con las muestras inicial y final de la medición. De esta forma, tanto el instante de inicio de la partida como los tiempos de respuesta influyen en las selecciones posteriores.

4.5. Validación de respuestas y avance

El bucle `wait_button` permanece activo mientras el valor leído desde los botones sea cero. Cuando aparece un valor distinto de cero, el programa lo compara con la máscara del LED objetivo almacenada en `x4`.

Si ambos valores coinciden, el programa salta a la etiqueta `correct`. Cualquier otro valor se considera una respuesta incorrecta y provoca el reinicio de la ronda. Una respuesta incorrecta no modifica `x9` ni `x11`; por lo tanto, los aciertos y tiempos válidos acumulados anteriormente se conservan.

Al detectarse una respuesta incorrecta se reinicia directamente la fase de preparación.

4.6. Medición y conversión del tiempo de reacción

La medición se realiza utilizando el contador de ciclos. Cuando se enciende el LED objetivo, el programa obtiene una primera lectura del contador. Al detectar el botón correcto se obtiene una segunda lectura.

$$N_{\text{ciclos}} = C_{\text{final}} - C_{\text{inicial}}$$

Considerando una frecuencia de reloj de 25 MHz, una décima de segundo corresponde a:

$$25\,000\,000 \cdot 0{,}1 = 2\,500\,000 \text{ ciclos}$$

Para realizar la conversión sin utilizar una instrucción de división, el programa resta repetidamente la constante 2 500 000 de la diferencia de ciclos. Cada resta representa una décima de segundo transcurrida.

$$q = \left\lfloor \frac{N_{\text{ciclos}}}{2\,500\,000} \right\rfloor$$

El valor q corresponde al tiempo expresado en décimas de segundo. La conversión es entera, por lo que los resultados se truncan.

4.7. Visualización de resultados

El programa construye un valor compatible con los dos displays de 7 segmentos. Cuando las unidades alcanzan diez, se realiza un ajuste que permite representar correctamente dos dígitos decimales.

Al finalizar la conversión, el resultado se escribe en los displays. No se utiliza un punto decimal, por lo que un valor como `08` debe interpretarse como 0,8 segundos. El rango decimal representable correctamente es de 00 a 99 décimas.

4.8. Conteo de rondas y cálculo del promedio

Cuando termina una conversión válida, el tiempo almacenado en `x12` se suma al acumulador `x11`, mientras que `x9` se incrementa en una unidad.

Si aún no se han completado diez respuestas correctas, comienza una nueva ronda. Si `x9` alcanza diez, el programa pasa al cálculo del promedio final.

$$S = q_1 + q_2 + \cdots + q_{10}$$

$$Q_{\text{prom}} = \left\lfloor \frac{S}{10} \right\rfloor$$

La división por diez se implementa mediante restas sucesivas. El promedio mostrado es un valor entero truncado en décimas de segundo.

5. Desarrollo del Assembler

5.1. Objetivo y procesamiento del archivo

El archivo `assembler.py` permite generar el programa del juego sin recurrir a un assembler RISC-V externo. Utiliza únicamente los módulos `sys` y `re` de Python. Si no se entregan argumentos, utiliza `game.s` como archivo de entrada y genera `game.hex`; también permite especificar rutas de entrada y salida desde la línea de comandos.

El procesamiento se realiza en dos pasadas. La función `clean_line` elimina comentarios y espacios exteriores, mientras que `first_pass` ignora líneas vacías, registra las etiquetas y asocia a cada instrucción una dirección y su número de línea.

La dirección inicial es cero y cada instrucción aumenta el PC en cuatro bytes, lo que permite resolver referencias a etiquetas ubicadas posteriormente.

Posteriormente, `assemble_instruction` separa los operandos, identifica el mnemónico y llama al codificador correspondiente. Los errores se detectan antes de escribir el resultado final, evitando generar un archivo incompleto si alguna instrucción es inválida.

5.2. Sintaxis admitida e inmediatos

El assembler reemplaza las comas por espacios y separa los operandos mediante espacios en blanco. Los mnemónicos se convierten a minúsculas.

Los registros deben escribirse con el prefijo `x` seguido de un número entre 0 y 15. Los inmediatos pueden escribirse en formato decimal o hexadecimal con prefijo `0x`.

Los operandos de memoria deben escribirse con la forma `desplazamiento(xN)`, por ejemplo `12(x2)`.

Las líneas que comienzan con punto, como `.section` o `.text`, se omiten y no generan palabras de código máquina.

5.3. Instrucciones soportadas

El assembler soporta las instrucciones utilizadas por el juego: `lui`, `addi`, `andi`, `add`, `sub`, `xor`, `and`, `or`, `lw`, `sw`, `beq`, `bne`, `bge` y `jal`.

5.4. Procesamiento de etiquetas y saltos

Una etiqueta termina con dos puntos y puede aparecer sola o antes de una instrucción. Durante la primera pasada, el assembler registra cada etiqueta junto con su dirección.

Para un destino simbólico se calcula:

$$\text{offset} = \text{dirección de la etiqueta} - PC_{\text{actual}}$$

Esto permite utilizar etiquetas tanto hacia adelante como hacia atrás dentro del programa.

5.5. Codificación de las palabras de 32 bits

Cada codificador construye una palabra de 32 bits combinando los campos correspondientes al formato de la instrucción. Se implementan los formatos R, I, S, B, U y J utilizados por las instrucciones del juego.

En los formatos S, B y J los bits del inmediato se reorganizan según la codificación correspondiente. En los saltos, el bit menos significativo no se almacena porque los desplazamientos válidos deben estar alineados.

5.6. Ejemplo de traducción Assembly a código máquina

Se considera la instrucción:

```
lw x7, 12(x2)
```

Esta instrucción utiliza formato I. Sus campos son:

Campo	Bits	Valor	Binario
Inmediato	31:20	12	`000000001100`
Registro base	19:15	2	`00010`
funct3	14:12	2	`010`
Registro destino	11:7	7	`00111`
Opcode	6:0	`0x03`	`0000011`

Cuadro 3: Campos formato I.

Fuente: Elaboración propia

Al concatenar los campos se obtiene:

```
00000000110000010010001110000011
```

cuya representación hexadecimal es:

```
0x00C12383
```

La palabra coincide con el resultado generado por `assembler.py` en `game.hex`.

5.7. Validación y pruebas del assembler

Para validar el assembler se verificó que los registros utilizados pertenecieran al rango `x0--x15` y que los inmediatos se encontraran dentro del rango permitido para cada formato.

También se verificó el procesamiento de etiquetas utilizadas como destino de branches y saltos, comprobando que el desplazamiento se calcule respecto del PC de la instrucción correspondiente.

Finalmente, se utilizó el assembler para traducir el programa completo `game.s`. El proceso generó correctamente `game.hex`, con una palabra hexadecimal de 32 bits por instrucción. El archivo resultante se utilizó posteriormente para sintetizar y ejecutar el juego en la FPGA.

6. Integración y proceso de compilación

6.1. Modificaciones del Makefile

El archivo `Makefile` se encarga de automatizar las distintas etapas necesarias para compilar e implementar el proyecto en la FPGA. Se modificó para incorporar `game_top` como módulo superior y generar automáticamente `game.hex` desde `game.s` utilizando `assembler.py`.

De esta manera, el proceso de compilación integra tanto el hardware desarrollado en Verilog como el programa en Assembly. Las dependencias establecidas permiten regenerar el archivo hexadecimal cuando se modifica el código fuente del juego.

6.2. Generación automática, síntesis y programación

El flujo implementado es:

`game.s` → `assembler.py` → `game.hex` → Yosys → nextpnr-ice40 → icepack → iceprog

El comando `make game` genera `game.hex` cuando sus dependencias lo requieren. El comando `make` ejecuta el flujo completo hasta la programación de la FPGA. El comando `make clean` elimina los archivos generados de síntesis, Place and Route y bitstream, pero conserva `game.hex`.

6.3. Interacción entre software y hardware

La memoria inicializada constituye el punto de unión entre el software y el hardware. El programa Assembly determina las palabras almacenadas en memoria, mientras que el hardware les da significado mediante el procesador Espino Core y las direcciones MMIO.

Para que una modificación en `game.s` llegue a la FPGA, se debe regenerar `game.hex` y construir un nuevo bitstream.

7. Validación y pruebas

7.1. Validación del funcionamiento general

Para comprobar el funcionamiento del sistema integrado se realizaron pruebas directamente sobre la FPGA Lattice iCE40 HX1K.

Se verificó que el sistema espera la pulsación inicial del usuario, inicia la primera ronda, enciende los cuatro LEDs durante la fase de preparación y posteriormente deja encendido únicamente el LED objetivo.

Cuando el botón presionado corresponde al LED seleccionado, el programa acepta la respuesta y continúa con el cálculo del tiempo. Cuando el botón no coincide, la ronda se reinicia y las respuestas correctas acumuladas anteriormente se conservan.

También se comprobó que la selección del LED no sigue una secuencia fija entre partidas, ya que depende de valores obtenidos desde el contador de ciclos.

7.2. Validación de la medición del tiempo

La medición se verificó observando los valores mostrados en los displays después de una respuesta correcta.

Considerando una frecuencia de 25 MHz, una décima de segundo equivale a 2 500 000 ciclos. Durante las pruebas se verificó que el valor mostrado cambia de acuerdo con el tiempo transcurrido entre el encendido del LED y la pulsación del botón.

Por ejemplo, `08` corresponde aproximadamente a 0,8 segundos.

7.3. Validación de las diez rondas y promedio final

Se verificó el funcionamiento completo de una partida de diez respuestas correctas. Cada respuesta válida incrementa `x9` y suma el tiempo obtenido al acumulador `x11`.

Al completar la décima respuesta, el sistema pasa al cálculo del promedio final. Durante las pruebas se comprobó que el tiempo de la décima ronda permanece visible brevemente antes de que el display sea actualizado con el promedio de las diez respuestas.

7.4. Validación de la integración hardware-software

Finalmente, se verificó el flujo completo desde el programa Assembly hasta la ejecución en la FPGA. El archivo `game.s` se traduce mediante `assembler.py` para generar `game.hex`. Posteriormente, el Makefile utiliza este archivo para sintetizar el diseño con Yosys, realizar el Place and Route con `nextpnr-ice40`, generar el bitstream mediante `icepack` y programar la FPGA utilizando `iceprog`.

Después de programar la placa se comprobó el correcto control de los LEDs, la lectura de los botones, la visualización de los tiempos y el cálculo del promedio final.

8. Declaración de uso de IA

En el presente informe se utilizó inteligencia artificial como apoyo para la redacción, la corrección ortográfica y la organización del contenido.